Bidirectional Salesforce ↔ PostgreSQL Synchronization

Tequity — Round 3 System Design Assignment

Document status: Proposed architecture; not implemented or production-verified.

1. Assumptions and Clarifying Questions

1.1 Problem and scale

Tequity's PostgreSQL-backed portal and a client's Salesforce CRM independently create and update customer and contact records. Changes must synchronize bidirectionally without losing accepted edits, silently discarding conflicts, or generating infinite update loops.

The system must handle:

- Approximately **5 million customer and contact records**.
- A strict **30-second end-to-end synchronization requirement** during healthy operation.
- A Salesforce API limit of **100 requests/second**.
- Duplicate, out-of-order, and silently missing Salesforce webhooks.
- Concurrent edits in both systems.
- Six-hour Salesforce outages while the portal continues accepting edits.
- Preservation of edit history, including losing conflict evidence.

For capacity planning, assume a peak of **25 Salesforce edits/second and 25 portal edits/second**, with approximately five combined edits/second during quieter periods.

A six-hour outage at 25 portal edits/second produces **540,000 logical outbound obligations**, potentially involving repeated edits to the same records.

1.2 Records and interfaces

Customers contain name, status, and owner. Contacts contain full name, phone, email, city, owner, and a parent customer relationship.

Tequity records use UUIDs; Salesforce records use independent Salesforce IDs. Durable mappings connect the two identities.

Salesforce webhooks are assumed to provide eventId, recordId, objectType, occurredAt, changedBy, and changedFields. Events contain changed fields rather than necessarily providing complete records.

Portal edits use a PostgreSQL-managed portal_version for optimistic concurrency. A stale version produces HTTP 409 Conflict.

1.3 Unverified Salesforce dependencies

The design depends on the following capabilities, none of which is established as verified:

1. **Complete historical changes:** A durable historical source exposes every relevant intermediate edit, changed fields, actor, timestamp, and resumable cursor, retained beyond the outage and recovery window.
2. **Reliable ordering:** Live and historical events expose compatible, trustworthy per-record source ordering/version semantics.
3. **Retry-safe creates:** Unique external-ID and compatible upsert behavior support safe identity establishment, with verification of ambiguous outcomes.
4. **Cross-system causal evidence:** Sufficient baseline or acknowledgment evidence distinguishes sequential edits from genuinely concurrent edits.

5. **Bounded visibility and API behavior:** Change visibility, webhook delivery, API execution, queueing, and processing delays can be bounded for strict SLA analysis.
6. **Stable cross-channel event identity:** The same logical Salesforce edit can be identified consistently in live webhooks and historical replay.

If the final dependency is unavailable, a separately validated canonical event-identity strategy is necessary. Without reliable cross-channel identity, deduplication across live and historical sources cannot be guaranteed.

1.4 Conflict and correctness policy

Conflicts are evaluated per field:

- Independent changes to different fields should both survive.
- Established sequential edits follow causal order.
- Genuinely concurrent same-field edits use a provisional field-authority rule.
- Losing edits remain represented in immutable audit evidence.
- Unknown causality results in reconciliation rather than an arbitrary winner.

The provisional policy gives Salesforce authority over customer name/status/owner and contact name/phone/email/owner/parent relationship. Tequity has authority over contact city.

Business approval of this policy is required. Authority never automatically overrides an established later sequential edit.

Audit records preserve all available actor, timestamp, changed-field, and conflict evidence. Unknown prior values remain explicitly unknown; the system does not claim to reconstruct information never supplied by Salesforce.

1.5 Guarantee boundaries

The design distinguishes three properties:

- **Durability:** Accepted portal edits and durably received Salesforce events are retained in PostgreSQL.
- **Eventual recovery:** Pending work can be reconciled and replayed when required Salesforce history and API capabilities are available.
- **Strict latency:** Completion within 30 seconds requires bounded external delivery and processing behavior.

A successful portal write or webhook acknowledgment establishes durable local acceptance, not completed remote synchronization.

A strict 30-second Salesforce synchronization deadline is impossible while Salesforce is unavailable. Recovery after an outage may also exceed that deadline.

1.6 Questions for the client

Before production commitment, confirm:

1. Does Salesforce expose complete historical changes, including intermediate edits?
2. Are ordering and event identity consistent across live and historical delivery?
3. Are unique external-ID upserts and ambiguous-write verification supported?
4. How is the SLA treated during outages and recovery barriers?
5. What causal evidence is available for cross-system edits?
6. Is the proposed field-authority policy approved?
7. Which Salesforce operations consume the 100 requests/second quota?
8. What are the historical retention period and maximum visibility delay?

2. Systems, Callers, and Interfaces

2.1 Components

Salesforce CRM independently accepts customer and contact edits, sends webhook notifications, and exposes write and historical-change interfaces subject to verification.

Tequity Portal API handles local creates and updates, performs optimistic concurrency checks, and commits business changes, audit records, and outbound obligations atomically.

Webhook API validates Salesforce events and commits them to a durable PostgreSQL inbox before acknowledging receipt.

Inbound workers apply received events using field-level ordering, causal reconciliation, and atomic business transactions.

Outbound workers deliver portal changes using per-record FIFO, a global API limiter, durable attempts, and remote confirmation evidence.

Recovery workers retrieve missing Salesforce history, maintain durable checkpoints, reconcile events, and manage recovery barriers.

PostgreSQL stores business records, mappings, inbox/outbox state, per-field synchronization state, audit evidence, idempotency records, and recovery metadata.

2.2 Logical architecture

[Insert approved high-level architecture diagram during Deliverable D2.]

Inbound:

Salesforce → Webhook API → PostgreSQL Inbox → Inbound Worker → Portal Business Tables

Outbound:

Portal Client → Portal API → Business Tables + Audit + Outbox → Outbound Worker → Salesforce API

Recovery:

Salesforce Historical Source → Recovery Worker → Checkpoint + Inbox → Reconciliation → Barrier Release

Shared state includes identity mappings, field synchronization state, confirmed outbound values, and immutable audit history.

The final diagram will show the global API limiter, per-record worker lanes, and recovery barriers.

2.3 Interfaces

Interface	Caller → Receiver	Responsibility
Portal create/update API	Portal client → Tequity	Validate edit, enforce optimistic concurrency, commit edit and outbox
Salesforce webhook	Salesforce → Tequity	Validate, deduplicate, durably accept event
Salesforce write API	Outbound worker → Salesforce	Create/upsert/update under shared rate limit
Salesforce historical API	Recovery worker → Salesforce	Retrieve missing changes and source cursors
PostgreSQL worker operations	Workers → PostgreSQL	Claim, reconcile, confirm, and checkpoint

Webhook ingress is treated separately from Salesforce API quota usage. Relevant outbound calls, retries, reads, and verification requests consume the applicable quota.

2.4 Delivery and transaction boundaries

Delivery is **at least once with idempotent processing**, not exactly-once transport.

External Salesforce calls occur outside PostgreSQL transactions. Durable obligations, leases, fencing tokens, attempt history, and verification protect recovery from crashes and ambiguous outcomes.

A PostgreSQL transaction cannot atomically commit a remote Salesforce API operation.

3. Subproblems in Implementation Order

Order	Subproblem	Outcome
1	Business data and identity mappings	Stable portal/Salesforce identity and parent dependencies
2	Durable capture	Atomic portal edit + audit + outbox; durable webhook inbox
3	Background delivery	Claimable work, retry scheduling, crash recovery
4	Ordering and idempotency	Per-record FIFO, event deduplication, portal request idempotency
5	Field-level reconciliation	Causal comparison, independent-field merge, conflict evidence
6	Salesforce delivery controls	Rate limiting, safe retries, confirmed-field state
7	Loop prevention	Origin-aware processing and controlled propagation
8	Historical recovery	Checkpoints, missing-event replay, recovery barriers
9	Validation and monitoring	Failure scenarios, capacity, SLA analysis, alerts

The fundamental dependency is **durability before delivery**: every asynchronous responsibility must first be represented by committed PostgreSQL state.

Safe recovery also depends on established identity, ordering, idempotency, and reconciliation rules.

An intermediate implementation milestone must not be represented as satisfying all production guarantees.

4. Stored Data

PostgreSQL separates business state, logical synchronization obligations, external delivery attempts, confirmed remote state, and immutable audit evidence.

4.1 Business tables

customers and **contacts** store portal business fields, UUID primary keys, portal_version, and timestamps.

Contacts reference their parent customer through a foreign key.

Portal versions support optimistic concurrency and are not directly comparable to Salesforce source sequences.

4.2 Identity mappings

customer_identity_map and **contact_identity_map** map portal UUIDs to unique Salesforce IDs.

Mappings distinguish pending, confirmed, and reconciliation-required identity states.

Outbound contact creation waits for the parent customer's confirmed Salesforce mapping. Mutable fields such as name and email are not identity keys.

4.3 Salesforce inbox

salesforce_inbox stores webhook and historical events, including:

- Unique logical event identity.
- Object type and Salesforce record ID.
- Source sequence, timestamp, actor, and changed fields.
- Original payload.
- Processing status, retry schedule, attempts, lease token, and expiry.

Live and historical events use the same processing pipeline.

Cross-channel deduplication depends on the unverified stable event-identity contract described in Section 1.

4.4 Salesforce outbox

salesforce_outbox stores one logical synchronization obligation per accepted portal edit requiring outbound delivery.

Each obligation records object type, portal UUID/version, operation, changed fields, status, attempts, retry schedule, and lease metadata.

The approved uniqueness model is one obligation per object type, portal record, and portal version.

An outbox obligation represents required synchronization, not an individual Salesforce HTTP attempt.

4.5 Field-level synchronization state

customer_field_sync_state and **contact_field_sync_state** track source ordering, portal versions, causal baselines, and resolution status per record and field.

Separate **customer_confirmed_outbound_fields** and **contact_confirmed_outbound_fields** tables record field values and versions actually confirmed at Salesforce.

Pending or attempted values must not be mistaken for confirmed remote state.

4.6 Audit history

audit_history is append-only, with one entry per logical edit containing available actor, timestamps, source identity, and complete available changed-field old/new values.

Conflict resolutions produce separate append-only evidence describing competing edits, selected values, losing values, and resolution rules.

If an event lacks prior values or intermediate history, those facts remain explicitly unknown. Complete reconstruction depends on the verified historical source.

4.7 Portal idempotency

portal_idempotency_records stores a scoped key, request hash, and original HTTP response status/body atomically with the portal edit.

Identical retries replay the original response. Reuse with different content is rejected.

4.8 Salesforce delivery attempts

salesforce_outbound_attempts stores each external request's actual payload, attempt number, request ID, timestamps, response evidence, and known or ambiguous outcome.

This distinguishes one logical edit from multiple delivery attempts.

4.9 Recovery tables

The recovery model contains:

- **recovery_checkpoints** — durable historical cursor and run status.
- **recovery_obligations** — outstanding completeness and reconciliation requirements.
- **recovery_quarantine** — events or conditions requiring investigation.
- **recovery_quarantine_obligations** — many-to-many links between quarantined evidence and obligations.

A checkpoint proves durable ingestion progress, not completed business application.

Per-record recovery barrier release requires evidence of completeness for the record and its relevant dependencies. If such evidence cannot be established, a broader or global barrier remains active.

4.10 Data invariants

1. A committed portal edit has its required audit and outbox entries.
2. A successfully acknowledged Salesforce event is durably recorded.
3. Duplicate logical events do not produce duplicate business effects.
4. Source-specific versions are not treated as globally comparable.
5. Attempted outbound values are not treated as confirmed values.
6. Confirmed obligations have durable confirmation evidence.
7. Ambiguous writes are reconciled before unsafe retries.
8. Historical checkpoints never advance before durable page ingestion.
9. Losing edits and unresolved conflicts retain available audit evidence.
10. Recovery barriers remain until relevant completeness requirements are satisfied.

Full DDL, indexes, worker-claim SQL, and transaction pseudocode belong in the technical appendix.

5. Inbound Synchronization: Salesforce → Tequity

5.1 Webhook acceptance

The webhook handler:

1. Validates the request and required event fields.
2. Inserts the event into salesforce_inbox.
3. Recognizes an identical previously accepted event.
4. Commits the inbox transaction.
5. Returns HTTP 2xx only after durable persistence.

An acknowledgment means **accepted for processing**, not applied to the portal.

Conflicting payloads for an existing event identity create an integrity exception.

5.2 Atomic inbound processing

An inbound worker claims eligible work using short PostgreSQL transactions, FOR UPDATE SKIP LOCKED, and durable lease/fencing metadata.

The application transaction then:

1. Verifies current lease ownership.
2. Locks the affected business record.
3. Locks affected field-state rows in deterministic order.
4. Evaluates Salesforce ordering and cross-system causality.
5. Applies eligible fields and resolves established conflicts.
6. Appends audit and resolution evidence.

7. Updates field synchronization state.
8. Marks the event processed or explicitly unresolved.
9. Commits.

A crash before commit rolls back the business change, audit, and completion together.

5.3 Identity and parent dependencies

An event referencing an unknown Salesforce identity is held until a safe mapping can be established.

Contacts cannot be associated with an unknown parent customer. Missing dependencies remain pending rather than being guessed.

5.4 Per-field ordering and causality

Each changed field is evaluated independently.

An older Salesforce event may still contain a valid update to a field untouched by newer Salesforce events.

However, **passing the Salesforce per-field sequence check is necessary but not sufficient**.

The worker must also evaluate cross-system causal state. An older Salesforce update must not overwrite a causally later portal edit merely because its Salesforce field sequence is eligible.

Genuinely concurrent same-field changes use the approved authority policy. Unknown causality results in reconciliation.

5.5 Missing webhooks

Historical Salesforce changes enter the same inbox and deduplication pipeline.

Historical ingestion and checkpoint advancement are atomic.

Complete recovery of silently missing intermediate edits requires a verified, sufficiently complete historical source and reliable cross-channel event identity.

6. Outbound Synchronization: Tequity → Salesforce

6.1 Portal edit transaction

The portal API commits the following in one PostgreSQL transaction:

1. Claim or validate the scoped idempotency key and request hash.
2. Check the expected portal version.
3. Apply the business edit and increment portal_version.
4. Append immutable audit evidence.
5. Insert the outbound obligation.
6. Store the original HTTP response.
7. Commit.

A stale version produces HTTP 409 Conflict without applying the stale edit.

A successful response establishes local durability, not remote synchronization.

6.2 Claiming and per-record FIFO

Workers use short FOR UPDATE SKIP LOCKED claims, durable lease tokens, and lease expiry.

The earliest unresolved obligation for a record must be resolved before a later obligation is sent.

Independent records may be processed concurrently.

A row lease alone does not serialize remote effects. The design requires explicit per-record lane ownership, including protection against a second worker sending a later version while an earlier attempt

remains ambiguous or in flight.

An unresolved ambiguous attempt blocks unsafe progression of that record's FIFO lane.

External API calls occur outside PostgreSQL transactions. Fencing prevents stale database completion but cannot cancel a Salesforce request already in flight.

The exact lane-ownership mechanism remains an implementation detail requiring specification.

6.3 API rate limiting

The Salesforce API limit is 100 requests/second. The design provisionally operates at **90 requests/second**.

Category	Initial share
Live synchronization	50 requests/s
Recovery	30 requests/s
Retry/reconciliation	10 requests/s
Total	**90 requests/s**

The scheduler is weighted-fair and work-conserving: unused allocation may be borrowed.

All applicable Salesforce writes, reads, retries, and verification calls consume the shared quota.

Concurrency is bounded separately from request rate.

6.4 Creates and dependencies

Retry-safe creates assume unique Salesforce external-ID and upsert semantics, which remain unverified.

Outbound contact creation waits for the parent's durable Salesforce mapping.

Unknown create outcomes are reconciled before potentially duplicating the remote record.

6.5 Retries and ambiguous outcomes

Known transient failures use bounded exponential backoff with jitter.

Permanent failures remain visible for remediation.

An ambiguous outcome is neither assumed successful nor treated as a safe failure to retry.

The system uses durable attempt evidence and supported Salesforce verification to establish the outcome before proceeding.

6.6 Confirmation transaction

After sufficient Salesforce acceptance evidence, the worker performs a fenced PostgreSQL transaction:

1. Verify lease ownership.
2. Record attempt outcome and evidence.
3. Insert or validate the Salesforce ID mapping.
4. Update confirmed field values using the actual payload sent.
5. Advance confirmed versions monotonically.
6. Mark the obligation CONFIRMED.
7. Commit.

A remote Salesforce write and its PostgreSQL confirmation are not transactionally atomic.

7. Concurrent Edit Conflict Resolution

7.1 Independent fields

Different-field changes should both survive.

For example, if Salesforce updates a contact's phone while Tequity updates its city, the resulting record retains both changes unless additional causal evidence requires otherwise.

7.2 Sequential versus concurrent edits

Salesforce sequence numbers and portal versions belong to separate ordering domains.

The design uses causal baselines, confirmed outbound state, and acknowledgment evidence to distinguish edits that observed one another from genuinely concurrent edits.

Known sequential edits follow causal order.

Unknown causality is held for reconciliation rather than resolved through arbitrary timestamp comparison.

7.3 Same-field authority

For genuinely concurrent same-field edits:

Fields	Provisional authority
Customer name/status/owner	Salesforce
Contact name/phone/email/owner/parent	Salesforce
Contact city	Tequity

For example, if Salesforce independently changes contact city to Mumbai while Tequity changes it to Pune, Tequity's value wins under the provisional policy.

The losing Salesforce edit remains in audit history.

This policy requires business approval.

7.4 Audit and resolution evidence

Each logical edit has an append-only audit record containing all available changed-field old/new values, actor, and timestamps.

Conflict resolution generates separate append-only evidence of the competing edits, causal basis, selected winner, and losing value.

Unavailable prior values remain explicitly unknown.

7.5 Winner propagation

A winning conflict value may require deliberate propagation to the losing system.

That operation has its own durable, idempotent reconciliation identity and origin evidence.

It must not be treated as an ordinary new user edit that initiates another conflict loop.

If safe propagation cannot be established, the conflict remains unresolved.

8. Duplicate and Out-of-Order Webhooks

8.1 Duplicate events

The inbox enforces unique logical event identity.

An identical duplicate is acknowledged without creating additional business changes or audit entries.

A conflicting payload with the same identity creates an integrity exception.

Cross-channel deduplication depends on verified stable event identity or a separately validated equivalent strategy.

8.2 Out-of-order changes to one field

Suppose Salesforce sequence 201 changes phone to 111 and sequence 202 changes phone to 222.

If 202 is applied first, sequence 201 must not later restore the older value.

Per-field source sequence state prevents that regression, subject to cross-system causal checks.

8.3 Older changes to unrelated fields

Suppose sequence 201 changes city while sequence 202 changes phone.

Receiving 202 first does not automatically invalidate the city update in 201.

The worker evaluates each changed field's source ordering and cross-system causal eligibility independently.

8.4 Gaps and incomplete history

A source sequence gap may indicate missing events, depending on Salesforce's verified ordering contract.

Newer events do not automatically prove complete history.

Unresolved completeness concerns create recovery obligations and trigger historical reconciliation.

9. Infinite Loop Prevention

9.1 Origin-aware application

Applying a Salesforce-origin event to portal business data does not automatically create a portal-origin outbound obligation.

This behavior is enforced in the same transactional application path.

9.2 Confirmed state and echo detection

Confirmed outbound field values and versions help recognize Salesforce notifications reflecting previously delivered portal edits.

However, **matching field values alone do not prove an echo**.

Reliable suppression also requires origin, correlation, or causal confirmation evidence. An unrelated Salesforce user edit with the same value must not be incorrectly classified solely by equality.

9.3 Deliberate conflict propagation

A conflict-resolution write is a distinct idempotent obligation with stable logical identity and origin evidence.

Its resulting acknowledgment or echo must not create another equivalent reconciliation obligation.

The system distinguishes user-origin edits, replicated inbound edits, confirmed outbound echoes, and deliberate reconciliation writes.

9.4 Loop invariant

A synchronized change must not create an unbounded chain of equivalent updates between Salesforce and Tequity.

If origin or causal evidence is ambiguous, the operation remains unresolved rather than triggering another potentially looping write.

10. Six-Hour Salesforce Outage, Recovery, and Capacity Analysis

10.1 Outage behavior

During Salesforce unavailability:

1. The portal continues accepting valid edits.
2. Business changes, audit records, and outbound obligations commit atomically.
3. Failed delivery remains pending with retry metadata.
4. Known transient failures use bounded backoff.
5. Ambiguous outcomes remain unresolved pending verification.
6. Recovery barriers prevent outbound replay that could overwrite unknown Salesforce changes.

A successful portal response indicates durable local acceptance, not Salesforce synchronization.

10.2 Recovery sequence

Step 1 — Establish the boundary. Record the historical source position or high-water mark and identify the recovery window, including overlap with resumed live delivery.

Step 2 — Retrieve history. Read Salesforce historical changes through the shared API limiter. Insert each page's events and advance its durable checkpoint atomically.

Step 3 — Reconcile. Process historical events through the normal inbox pipeline, using identity mapping, per-field ordering, causal conflict rules, and audit evidence.

Step 4 — Release barriers. Release affected per-record barriers only when relevant history and dependencies are proven complete. Otherwise retain a broader or global barrier.

Step 5 — Drain backlog. Resume safe outbound delivery using per-record FIFO, confirmation, and weighted-fair scheduling.

A checkpoint proves durable historical ingestion, not completed business application.

10.3 Recovery invariants

- Accepted portal edits remain durably represented until confirmed or explicitly unresolved.
- Historical page ingestion and checkpoint advancement are atomic.
- Live/historical duplicates do not create duplicate effects.
- Relevant Salesforce history is reconciled before unsafe conflicting outbound replay.
- Ambiguous writes are verified before unsafe retries.
- Recovery barriers require sufficient completeness evidence before release.
- Unresolved changes remain auditable and visible.

10.4 Outage backlog and API budget

At 25 portal edits/second for six hours:

6 × 60 × 60 = 21,600 seconds

25 × 21,600 = 540,000 logical outbound obligations

The operating Salesforce API budget is 90 requests/second, with provisional shares of 50 live, 30 recovery, and 10 retry/reconciliation requests/second.

These are scheduling allocations, not independent hard limits.

10.5 Ideal backlog clearance

For an illustrative lower-bound calculation, assume **exactly one successful Salesforce API request per backlog obligation**, no retries, no additional reads or verification calls, no dependency stalls, and no competing quota consumers within the stated effective backlog rate.

Then:

Clearance time = 540,000 ÷ effective backlog-processing requests/second

Effective backlog rate	Ideal clearance time
30 requests/s	5 hours
40 requests/s	3 hours 45 minutes
50 requests/s	3 hours
65 requests/s	Approximately 2 hours 18 minutes
75 requests/s	2 hours

The 65 requests/second example is an optimistic upper-bound allocation under the 90 requests/second budget after reserving 25 requests/second for live portal writes. It excludes Salesforce-originated work, historical reads, retries, verification, and other consumers.

The 75 requests/second example would use the full 100 requests/second cap alongside 25 live portal writes, leaving no headroom.

In practice, one obligation may require multiple API calls, and some obligations may be resolved through approved reconciliation without a corresponding write. Therefore, **logical obligation count is not necessarily equal to API request count**.

These calculations are illustrative capacity bounds, not guaranteed recovery completion times.

10.6 Strict 30-second SLA analysis

Synchronization latency is measured from the original source edit to confirmed destination application.

The approved illustrative healthy-operation budgets are:

Salesforce → Tequity

Stage	Target
Webhook delivery	10 s
Validation and inbox commit	2 s
Queue wait	8 s
Application/conflict transaction	5 s
Safety margin	5 s
Total	**30 s**

Tequity → Salesforce

Stage	Target
Portal edit and outbox commit	2 s

Stage	Target
Queue wait and claim	8 s
Salesforce API execution	10 s
Confirmation transaction	3 s
Safety margin	7 s
Total	**30 s**

These are illustrative targets, not verified worst-case bounds.

At 25 portal edits/second, nominal utilization relative to the 90 requests/second budget is approximately **27.8%**, assuming one API request per edit.

Aggregate headroom does not establish a strict deadline.

Per-record FIFO counterexample: Five pending edits to the same record, each requiring eight seconds of sequential remote processing, can leave the fifth waiting approximately 32 seconds before its own request begins.

Missing-webhook counterexample: A Salesforce edit that becomes visible through historical retrieval only after 60 seconds cannot meet a 30-second end-to-end deadline.

10.7 Feasibility conclusion

The design provides durable local acceptance and supports eventual recovery under its verified external dependencies.

It cannot unconditionally guarantee strict 30-second synchronization.

- **During a six-hour Salesforce outage:** The deadline is impossible for portal-to-Salesforce changes.
- **During recovery:** Backlog, historical reconciliation, and barriers may exceed 30 seconds.
- **During healthy operation:** Compliance requires verified worst-case bounds for change visibility, API latency, queueing, per-record dependencies, and retries.
- **For silently lost webhooks:** Complete intermediate-edit recovery requires complete historical evidence.
- **For ambiguous Salesforce writes:** PostgreSQL alone cannot provide distributed exactly-once execution.

The strict 30-second requirement remains visible as a contractual condition; it is not replaced by p99 latency.

11. Production Monitoring Metrics and Alert Thresholds

Exactly **three primary metrics** are defined.

Metric	Warning	Critical
M1 — End-to-end synchronization latency	20 seconds	30 seconds
M2 — Oldest unresolved obligation age	15 seconds	30 seconds
M3 — Unresolved correctness exceptions	At least 1	At least 10, or any confirmed corruption

11.1 M1 — End-to-end latency

Measure from the original source edit to confirmed application at the destination, separately for both directions.

Include unfinished edits whose age is increasing, not only completed edits.

A warning occurs at 20 seconds; a critical alert occurs at 30 seconds or for a completed edit measured above the deadline.

Maximum and p99 latency may be reported, but p99 does not replace the strict SLA.

Silently missing Salesforce events may remain invisible until historical reconciliation discovers them; latency monitoring alone cannot prove completeness.

11.2 M2 — Oldest unresolved obligation

Measure the age of the oldest pending, processing, retrying, blocked, or reconciliation-required obligation.

Use its original source-edit or obligation timestamp, not the latest retry timestamp.

Warning: older than 15 seconds. Critical: 30 seconds or older.

During a Salesforce outage, this metric is expected to remain critical. Incident context may distinguish outage-related violations from healthy-operation failures, but **the metric must not be hidden, suppressed as compliant, or reset**.

11.3 M3 — Correctness exceptions

Count distinct unresolved logical integrity or reconciliation issues, including:

- Quarantined events with insufficient causal evidence.
- Ambiguous Salesforce writes.
- Identity mapping conflicts.
- Historical completeness gaps.
- Conflicting event payloads for one identity.

Warning: one or more unresolved exceptions.

Critical: ten or more, or immediately upon confirmed irreversible data loss or synchronization corruption.

Repeated attempts do not count as separate logical exceptions. Correctly resolved conflicts remain in audit history but are not unresolved exceptions.

11.4 Supporting telemetry

Additional diagnostic measurements include Salesforce API request rate, HTTP 429 responses, retries, queue depth, checkpoint progress, lease expirations, recovery barriers, and reconciliation progress.

These are supporting signals, not additional primary metrics.

All thresholds are proposed operational defaults, not empirically verified production baselines.

12. Out of Scope and Remaining Limitations

The proposed solution focuses on reliable bidirectional customer and contact **creates and updates**.

The following are excluded from implementation scope:

- Record deletion, tombstones, and hard-delete propagation.
- Authentication UI, administration screens, and complete access-control implementation.
- Multi-region active-active deployment or second-region disaster recovery.
- Testing against a real Salesforce tenant.
- Full production load testing and empirically established worst-case latency.
- General-purpose synchronization of unrelated Salesforce object types.
- A complete administrative conflict-resolution interface or operational dashboard.

Transport authentication, webhook validation, and secure credential handling remain necessary production concerns even though a full security implementation is outside this assignment.

Unresolved implementation dependencies

The architecture requires further specification or verification of:

1. A concrete per-record worker-lane ownership mechanism, including expired leases and in-flight Salesforce calls.
2. Exact cross-system causal-evidence representation and field-state transitions.
3. Salesforce historical completeness, ordering, retention, and stable cross-channel event identity.
4. Safe external-ID upsert and ambiguous-write verification contracts.
5. Bounded source visibility and API behavior for strict SLA proof.
6. Business approval of the provisional field-authority policy.

These are explicit design dependencies, **not completed implementation or verified integration capabilities**.

Final guarantee boundary

The design durably captures accepted portal edits and received Salesforce events, supports idempotent processing, preserves available immutable conflict evidence, and provides controlled recovery after failures.

Complete recovery of silently missing intermediate edits depends on a verified historical source. Remote Salesforce writes are not transactionally atomic with PostgreSQL. Strict 30-second synchronization cannot be met during Salesforce unavailability and remains unproven as a worst-case healthy-operation guarantee.

The result is a proposed recoverable, auditable, rate-limited synchronization architecture with explicit correctness boundaries and external dependencies.